Git Quick Reference

Configurar · Staging & Commits · Branches · Remoto

CONFIGURAR
& iniciar repo

Configuración inicial

```
git config --global user.name 'Oscar'
```
nombre de autor

```
git config --global user.email 'a@b.c'
```
email de autor

```
git config --global core.editor code
```
editor por defecto

```
git config --global init.defaultBranch main
```
rama inicial

```
git config --list
```
ver configuración

Iniciar repositorio

- `git init` nuevo repo en carpeta actual
- `git init nombre-repo` nuevo repo en nueva carpeta
- `git clone <url>` clonar repo remoto
- `git clone <url> .` clonar en carpeta actual
- `git clone --depth 1 <url>` clonar solo último commit

Staging area

- `git status` estado del working tree
- `git status -s` estado formato corto
- `git add .` agregar todos los cambios
- `git add archivo.js` agregar archivo específico
- `git add -p` agregar cambios interactivo
- `git restore --staged .` quitar del staging
- `git diff` cambios no staged
- `git diff --staged` cambios staged vs último commit

Commits

- `git commit -m 'mensaje'` commit con mensaje
- `git commit -am 'msg'` add+commit tracked files
- `git commit --amend` modificar último commit
- `git commit --amend --no-edit` amend sin cambiar msg

Conventional Commits

- `feat:` nueva funcionalidad
- `fix:` corrección de bug
- `docs:` documentación
- `style:` formato, sin lógica
- `refactor:` sin feat ni fix
- `test:` añade tests
- `chore:` tareas de mantenimiento

BRANCHES
& merge

Gestionar ramas

- `git branch` listar ramas locales
- `git branch -a` listar todas (local+remota)
- `git branch nombre` crear rama
- `git checkout nombre` cambiar de rama
- `git checkout -b nombre` crear y cambiar
- `git switch nombre` cambiar de rama (moderno)
- `git switch -c nombre` crear y cambiar (moderno)
- `git branch -d nombre` borrar rama local (seguro)
- `git branch -D nombre` borrar rama local (forzar)
- `git branch -m old new` renombrar rama

Merge & Rebase

- `git merge nombre` merge a rama actual
- `git merge --no-ff nombre` merge con commit explícito
- `git merge --squash nombre` squash todos los commits
- `git rebase main` rebase sobre main
- `git rebase -i HEAD~3` rebase interactivo 3 commits
- `git cherry-pick <hash>` traer commit específico

Conflictos

```
# Después de un merge con conflicto:
git status  # ver archivos en conflicto
# Editar archivos — resolver <<<< ==== >>>>
git add .  # marcar como resuelto
git commit  # finalizar merge
# Para abortar el merge:
git merge --abort
```

Tags

- `git tag v1.0.0` tag ligero
- `git tag -a v1.0.0 -m 'msg'` tag anotado (recomendado)
- `git push origin v1.0.0` push tag específico
- `git push --tags` push todos los tags
- `git tag -d v1.0.0` borrar tag local

REMOTO
push & pull

Gestionar remotos

- `git remote -v` ver remotos configurados
- `git remote add origin <url>` agregar remoto
- `git remote set-url origin <url>` cambiar URL remoto
- `git remote remove origin` eliminar remoto
- `git remote rename old new` renombrar remoto

Push & Pull

- `git fetch origin` descargar sin merge
- `git fetch --all` descargar todos los remotos
- `git pull` fetch + merge
- `git pull --rebase` fetch + rebase (más limpio)
- `git push origin main` push a rama específica
- `git push -u origin main` push y configurar upstream
- `git push --force-with-lease` push forzado (seguro)
- `git push origin :rama` borrar rama remota
- `git push --delete origin rama` borrar rama remota

Flujo GitHub típico

```
# 1. Clonar
git clone <url> && cd repo
# 2. Crear rama de trabajo
git switch -c feat/mi-feature
# 3. Trabajar y commitear
git add -p && git commit -m 'feat:..'
# 4. Traer cambios de main
git fetch origin
git rebase origin/main
# 5. Push y Pull Request
git push -u origin feat/mi-feature
```

ERRORES COMUNES — Configuración, Staging y Remoto

git push --force en rama compartida
--force reescribe el historial remoto. Usar --force-with-lease para proteger.

git add . sin revisar qué se agrega
Usar git status y git diff primero. git add -p para control granular.

Commitear en main directamente
Trabajar siempre en ramas. main/master debe estar protegido en el remoto.

Mensajes de commit sin contexto
'fix bug' no ayuda. Conventional Commits: 'fix(auth): corrige token expirado'.

```
git init
git add . && git commit -m 'init'
git branch -M main
git push -u origin main
git log --oneline --graph
# historial visual
git diff HEAD~1
# cambios del último commit
```

HISTORIAL
& diff

git log — ver historial

- `git log` historial completo
- `git log --oneline` una línea por commit
- `git log --oneline --graph` grafo de ramas
- `git log --oneline -10` últimos 10 commits
- `git log -p` historial con diffs
- `git log --author='Oscar'` commits de un autor
- `git log --since='2 weeks ago'` por fecha
- `git log --grep='feat'` buscar en mensajes
- `git log main..feature` commits en feature no en main
- `git log -- archivo.js` historial de un archivo
- `git shortlog -sn` commits por autor

git diff — comparar

- `git diff` working vs staging
- `git diff --staged` staging vs último commit
- `git diff HEAD` working vs último commit
- `git diff main feature` comparar dos ramas
- `git diff HEAD~1 HEAD` últimos dos commits
- `git diff <hash1> <hash2>` entre dos commits
- `git show <hash>` ver un commit específico
- `git show HEAD:archivo.js` archivo en último commit

Buscar en el historial

- `git blame archivo.js` quién escribió cada línea
- `git bisect start` iniciar búsqueda binaria
- `git bisect bad` marcar commit actual como malo
- `git bisect good <hash>` marcar commit conocido bueno
- `git bisect reset` finalizar bisect
- `git grep 'patrón'` buscar en archivos tracked

DESHACER
cambios

Deshacer — working tree

- `git restore archivo.js` descartar cambios del archivo
- `git restore .` descartar todos los cambios
- `git restore --staged .` sacar del staging area
- `git clean -fd` borrar archivos untracked
- `git clean -n` preview de lo que borrará

Deshacer commits

```
# reset — reescribe historial:
git reset --soft HEAD~1
# ↑ deshace commit, mantiene staged
git reset --mixed HEAD~1
# ↑ deshace commit, mantiene working
git reset --hard HEAD~1
# ↑ deshace commit y cambios ⚠

# revert — seguro para ramas compartidas:
git revert HEAD
# ↑ crea un nuevo commit que deshace
git revert <hash>
git revert HEAD~3..HEAD
# ↑ revertir rango de commits
```

Reflog — recuperar todo

- `git reflog` historial de todos los HEAD
- `git checkout <hash>` ir a cualquier estado previo
- `git reset --hard <hash>` restaurar a estado previo

Regla: el reflog guarda TODO durante 90 días. Nada se pierde.

STASH
& herramientas

git stash — guardar temporalmente

- `git stash` guardar cambios no committed
- `git stash push -m 'msg'` stash con descripción
- `git stash list` ver todos los stashes
- `git stash pop` restaurar y borrar stash
- `git stash apply` restaurar sin borrar stash
- `git stash apply stash@{2}` restaurar stash específico
- `git stash drop` borrar último stash
- `git stash clear` borrar todos los stashes
- `git stash branch nombre` crear rama desde el stash

.gitignore

```
# Archivos y carpetas:
node_modules/
.env
.env.local
dist/
.next/
# Patrones:
*.log
*.tmp
**/__pycache__/
# Excepciones:
!.env.example
# Ignorar ya trackeado:
git rm --cached archivo
```

Alias útiles

```
git config --global alias.st status
git config --global alias.co checkout
git config --global alias.lg
  'log --oneline --graph --all'
# Uso: git lg
git config --global alias.undo
  'reset --soft HEAD~1'
# Uso: git undo
```

ERRORES COMUNES — Historial, Deshacer y Stash

git reset --hard sin backup
Primero git stash o crear una rama de backup. --hard borra cambios para siempre.

Rebase en rama compartida con otros
Rebase reescribe historia. En ramas compartidas usar merge. Rebase solo en local.

Commitear .env o secrets
Agregar .env al .gitignore ANTES del primer commit. Si ya está: git rm --cached.

Ignorar archivos después de haberlos committeado
.gitignore no funciona con archivos ya trackeados. Usar git rm --cached primero.